



GEM

Beps Smart Research

— • CURSO PRÁCTICO INTENSIVO • —

R Y RSTUDIO PARA EL ANÁLISIS DE DATOS

Fundamentos para Econometría



Manual práctico del estudiante



10 horas



5 sesiones de 2 horas



— Docente —

Mg. Miguel Jesús Armando Paredes Trujillo

Maestro en Ciencias Económicas y Economía Aplicada

Econometrista | Científico de Datos | Investigador Senior



Beps Smart Research

Material académico para uso formativo

Cómo usar este material

Este manual combina conceptos, definiciones, reglas de trabajo, ejemplos de código, ejercicios y casuísticas. La idea es que la parte teórica te permita comprender por qué se ejecuta cada instrucción y que la práctica convierta esa comprensión en una rutina de análisis reproducible.

Acceso a todos los recursos · Las bases, scripts de cada sesión y archivos de apoyo se encuentran en el repositorio oficial del curso en GitHub. Trabaja siempre con la versión publicada allí para mantener un único punto de acceso al material.

Repositorio oficial: <https://github.com/Econ-Paredes/R-Y-RSTUDIO-PARA-EL-ANALISIS-DE-DATOS>

1. Ingresa al repositorio del curso y revisa el README antes de iniciar.
2. Si trabajarás localmente, usa Code > Download ZIP, descomprime la carpeta y abre Curso_R_GEM.Rproj.
3. Mantén la estructura de carpetas. Evita mover bases o scripts fuera del proyecto mientras estés aprendiendo.
4. Lee primero la explicación de cada sección; luego copia o escribe el código en tu R Script y ejecútalo por bloques.
5. Cuando aparezca una nota, consejo, regla o error frecuente, úsalo como guía de buenas prácticas y no como un comando para memorizar.

Recurso visual	Qué significa
Definición	Concepto que debes comprender antes de ejecutar código.
Regla	Convención o práctica que conviene respetar durante todo el curso.
Buena práctica	Recomendación que mejora orden, reproducibilidad o seguridad del análisis.
Consejo / truco	Atajo o forma de trabajar con mayor fluidez.
Error frecuente	Situación habitual que genera resultados incorrectos o mensajes de error.
Ejercicio	Actividad para aplicar lo aprendido sin limitarse a repetir el ejemplo.

Resultado del curso

Al finalizar las 10 horas, el estudiante estará en condiciones de:

- Reconocer qué es R, qué es RStudio y cuál es la función de cada zona principal de la interfaz.
- Organizar un proyecto con carpeta raíz, subcarpetas, RStudio Project y rutas de trabajo consistentes.
- Distinguir Consola, R Script, R Markdown y Markdown, y seleccionar el formato adecuado según la finalidad.
- Comprender la lógica de objetos, asignación, operadores, funciones, tipos de datos, vectores, data frames y valores faltantes.
- Importar, inspeccionar, limpiar, transformar, resumir y unir bases mediante un flujo reproducible.
- Construir variables habituales en econometría y reconocer la estructura básica de datos de panel.
- Ejecutar una primera regresión con `lm()` comprendiendo que el modelo es otro objeto de R y que debe leerse dentro de un proceso analítico completo.

Ruta de 10 horas

Sesión	Núcleo de aprendizaje	Producto de la sesión
1	Entorno de trabajo + lógica de R + objetos, operadores, funciones, vectores, NA y data frames	Proyecto organizado y primer script reproducible
2	Importación, auditoría, limpieza, dplyr y transformación	Base depurada y documentada
3	Descriptivos, agrupación, joins, gráficos y correlación	Tabla de indicadores y visualizaciones
4	Transformaciones econométricas, factores, panel, rezagos y fórmulas	Base preparada para modelamiento
5	Pipeline completo y primera regresión con lm()	Mini proyecto integrado reproducible

Índice de contenidos

La numeración de páginas corresponde a esta edición del manual. Los títulos principales y subtítulos mantienen la misma secuencia que se utilizará en clase.

Contenido	Página
Resultado del curso	2
Ruta de 10 horas	3
ANTES DE INICIAR · Entender R, RStudio y organizar el trabajo	7
A. ¿Qué es R?	7
B. ¿Qué es RStudio?	7
C. La interfaz de RStudio: para qué sirve cada zona	7
D. Formas de trabajo: Consola, R Script, R Markdown y Markdown	8
E. La carpeta de trabajo: orden antes que código	9
F. Crear y abrir un RStudio Project	10
G. Paquetes: ampliar las capacidades de R	10
H. Comentarios: explicar el porqué del código	11
I. Ayuda y documentación dentro de R	11
SESIÓN 1 · Pensar en R	12
1. La lógica básica de programación	12
2. ¿Qué es un objeto en R?	12
3. Nombres de objetos: reglas y recomendaciones	13
4. Símbolos y operadores que debes reconocer	14
5. Operaciones y funciones	14
6. Tipos de datos básicos	15
7. Vectores: una columna de valores	16
8. Condiciones lógicas: la base de los filtros	16
9. NA: dato faltante	16
10. Data frame: la estructura central del análisis	17
11. Selección de filas y columnas	18
12. Matrices: un puente conceptual hacia econometría	18
Ejercicios de la sesión 1	18

Contenido	Página
SESIÓN 2 · De archivo bruto a base analítica	19
1. Importar no es “abrir”: es crear un objeto	19
2. Rutas relativas dentro del Project	19
3. Importar CSV, Excel y Stata	19
4. Auditoría inicial: nunca modelos inmediatamente después de importar	20
5. Identificadores y duplicados	20
6. Texto inconsistente y estandarización	21
7. El pipe <code> ></code> : leer una secuencia de transformaciones	21
8. Verbos fundamentales de dplyr	21
9. Limpieza reproducible	22
10. Crear variables para el análisis	22
Ejercicios de la sesión 2	22
SESIÓN 3 · Convertir datos en información	23
1. ¿Por qué describir antes de modelar?	23
2. Estadísticos descriptivos esenciales	23
3. Frecuencias y proporciones	23
4. <code>group_by()</code> + <code>summarise()</code> : de microdatos a indicadores	24
5. Llaves y joins: unir información sin perder control	24
6. Visualización con ggplot2: datos + estética + geometría	25
7. Correlación: dirección e intensidad lineal	25
Ejercicios de la sesión 3	26
SESIÓN 4 · Preparar variables para econometría	27
1. Transformar no es maquillar: cada variable necesita una razón	27
2. Logaritmos	27
3. Variables dummy	27
4. Términos cuadráticos	28
5. Factores y categoría de referencia	28
6. Interacciones: cuando una relación depende de otra variable	28

Contenido	Página
7. ¿Qué es una base de panel?	28
8. Rezagos y diferencias	29
9. Fórmulas de modelo en R	29
Ejercicios de la sesión 4	30
SESIÓN 5 · Del dato bruto a una primera regresión	31
1. El pipeline completo	31
2. ¿Qué hace <code>lm()</code> ?	31
3. Regresión múltiple: la fórmula crece, la lógica se mantiene	32
4. El modelo también es un objeto	32
5. Predicción didáctica	33
6. Diagnóstico visual introductorio	33
7. Exportar resultados	33
Proyecto integrador final	33
Hoja rápida de comandos	35
Errores frecuentes y cómo diagnosticarlos	36
Glosario mínimo	36
Referencias y recursos oficiales de consulta	37

ANTES DE INICIAR • Entender R, RStudio y organizar el trabajo

A. ¿Qué es R?

Definición • R es un lenguaje y un entorno para computación estadística y gráficos. Permite representar datos como objetos, aplicar funciones, automatizar procedimientos, producir gráficos y estimar modelos estadísticos y econométricos.

Pensar en R implica dejar de trabajar únicamente con clics y aprender a expresar una tarea mediante instrucciones que pueden guardarse, revisarse y ejecutarse nuevamente. Por eso, el valor de R no está solo en “hacer cálculos”: está en convertir el análisis en un proceso explícito y reproducible.

Idea	En R significa
Dato	Información que puede almacenarse: un número, texto, fecha, categoría, etc.
Objeto	Una estructura que R mantiene en memoria y a la que normalmente asignamos un nombre.
Expresión	Una instrucción que R interpreta y evalúa.
Función	Una operación reutilizable que recibe argumentos y devuelve un resultado.
Script	Archivo de texto donde guardamos instrucciones de R en un orden lógico.
Resultado	Puede ser un número, tabla, gráfico, archivo, modelo u otro objeto.

Regla • R distingue mayúsculas y minúsculas. Para R, ingreso, Ingreso e INGRESO son nombres diferentes.

B. ¿Qué es RStudio?

Definición • RStudio es un entorno de desarrollo integrado (IDE). No reemplaza a R: organiza el trabajo alrededor de R mediante un editor de código, consola, exploración de objetos, archivos, gráficos, paquetes, ayuda y otras herramientas.

Una forma útil de recordarlo es: R es el motor; RStudio es el tablero de control. El código finalmente lo interpreta R, mientras RStudio facilita escribirlo, ejecutarlo, guardarlo y observar lo que ocurre.

C. La interfaz de RStudio: para qué sirve cada zona

La distribución puede modificarse, pero la configuración habitual contiene cuatro paneles principales. En este curso utilizaremos especialmente Source, Console, Environment y Output.

Zona / pestaña	Finalidad	Cuándo la usarás
Source / Script	Editor donde escribes y guardas archivos .R, .Rmd, .qmd y otros textos.	La mayor parte del curso. Aquí debe vivir tu código reproducible.
Console	Ejecuta instrucciones de forma interactiva y muestra resultados, mensajes, advertencias y errores.	Pruebas rápidas y ejecución del código enviado desde Source.
Environment	Muestra los objetos creados en la sesión actual: vectores, tablas, funciones, modelos, etc.	Para comprobar qué objetos existen y, de forma orientativa, su tamaño o estructura.
History	Registra comandos ejecutados durante la sesión.	Para recuperar una instrucción; no sustituye al script guardado.
Files	Explora carpetas y archivos del proyecto o directorio.	Para verificar rutas, archivos disponibles y estructura del proyecto.
Plots	Muestra gráficos generados en la sesión.	Para revisar visualizaciones antes de exportarlas.
Packages	Permite ver, cargar y administrar paquetes instalados.	Como alternativa visual a <code>install.packages()</code> y <code>library()</code> .
Help	Muestra documentación de funciones y paquetes.	Cuando necesites conocer argumentos, ejemplos o detalles de una función.
Viewer	Muestra contenido HTML y aplicaciones interactivas.	En productos que generen salidas web; uso secundario en este curso.
Terminal	Ejecuta comandos del sistema operativo.	Uso complementario; no es necesario para aprender las bases de R.

Consejo · Usa Ctrl+1 para llevar el foco a Source y Ctrl+2 para ir a Console. Ctrl+L limpia visualmente la consola; no elimina los objetos del Environment.

D. Formas de trabajo: Consola, R Script, R Markdown y Markdown

Formato	Qué contiene	Ventaja principal	Uso recomendado en este curso
Console	Comandos interactivos sin un archivo que los preserve.	Rapidez para probar una instrucción.	Solo pruebas rápidas. No construir el análisis completo allí.
R Script (.R)	Código R ordenado en líneas y bloques, con comentarios.	Ligero, claro, escalable y sencillo de ejecutar por secciones.	Formato principal para limpieza, transformación y análisis local.

Formato	Qué contiene	Ventaja principal	Uso recomendado en este curso
R Markdown (.Rmd)	Texto en Markdown + bloques de código R + resultados.	Integra narrativa, código y resultados en un documento reproducible.	Útil para informes; se presenta como alternativa, no como flujo principal.
Markdown (.md)	Texto con sintaxis ligera de títulos, listas, enlaces y código, pero sin ejecutar R por sí mismo.	Excelente para documentación, por ejemplo un README de GitHub.	Para guías y documentación del proyecto.

Recomendación del curso · Trabajaremos principalmente en R Script (.R). Para un flujo local de análisis es ligero, transparente, fácil de depurar y permite separar datos, código y resultados. R Markdown será útil cuando el objetivo sea comunicar el análisis en un documento que combine narrativa y resultados.

Nota · RStudio también trabaja con Quarto (.qmd). Es una alternativa moderna para documentos reproducibles, pero no forma parte del núcleo de estas 10 horas para evitar sobrecargar el aprendizaje inicial.

E. La carpeta de trabajo: orden antes que código

Antes de importar una base conviene saber dónde está trabajando R. Si no controlas la ubicación, aparecen errores como “archivo no encontrado”, duplicación de archivos o resultados guardados en carpetas inesperadas.

```
# ¿Dónde está trabajando R en este momento?
getwd()

# ¿Qué archivos y carpetas ve R en esa ubicación?
list.files()
```

Qué observar · `getwd()` devuelve la ruta del directorio de trabajo actual. Ese directorio funciona como punto de referencia para las rutas relativas que uses después.

Si estás trabajando sin un Project y deseas redirigir temporalmente la sesión a una carpeta creada por ti, puedes utilizar `setwd()`. En Windows conviene usar / en lugar de \ dentro de la ruta.

```
# Ejemplo ilustrativo en Windows
setwd("D:/CURSO_R_GEM")
getwd()
```

Buena práctica · Primero aprende `getwd()` y `setwd()` para comprender el concepto de directorio de trabajo. Después usa RStudio Project como método principal: al abrir el archivo .Rproj, RStudio establece automáticamente como directorio de trabajo la raíz del proyecto.

Estructura recomendada del curso:

```
Curso_R_GEM/
├── Curso_R_GEM.Rproj
├── datos/
├── scripts/
├── resultados/
└── graficos/
```

Carpeta	Qué debe contener	Regla
datos	Bases originales de trabajo.	No modificar manualmente una base original si puedes reproducir el cambio con código.
scripts	Archivos .R ordenados por sesión o etapa del análisis.	Usar nombres cortos, claros y con numeración si existe una secuencia.
resultados	Tablas o archivos procesados generados por el análisis.	Deben poder regenerarse ejecutando el código.
graficos	Figuras exportadas.	Usar nombres que describan el contenido y, cuando corresponda, versión o fecha.

Regla de rutas · Si el Project está abierto y la base está en datos, utiliza "datos/archivo.csv". Evita escribir rutas absolutas muy largas como D:/Usuarios/.../archivo.csv en cada línea: dificultan mover el proyecto a otra PC.

F. Crear y abrir un RStudio Project

6. Crea una carpeta raíz para el curso o proyecto.
7. En RStudio selecciona File > New Project.
8. Si la carpeta ya existe, selecciona Existing Directory; si no existe, puedes crear un New Directory.
9. RStudio generará un archivo .Rproj. A partir de entonces, abre ese archivo para iniciar el trabajo.
10. Comprueba con getwd() que el directorio de trabajo corresponda a la raíz del proyecto.

Por qué importa · Un Project mantiene juntos código, datos, resultados y figuras y reduce la necesidad de cambiar manualmente el directorio en cada sesión.

G. Paquetes: ampliar las capacidades de R

R incorpora muchas funciones en su instalación base, pero gran parte del trabajo moderno se apoya en paquetes: colecciones de funciones, datos y documentación creadas para tareas específicas.

Acción	Qué significa	Frecuencia
install.packages("paquete")	Descarga e instala el paquete en tu computadora.	Normalmente una vez por instalación o cuando necesites actualizar.

Acción	Qué significa	Frecuencia
<code>library(paquete)</code>	Carga el paquete en la sesión actual para utilizar sus funciones.	Cada vez que inicies una nueva sesión que lo requiera.

```
# Instalar una vez
install.packages(c("tidyverse", "readxl", "haven"))

# Cargar en cada sesión
library(tidyverse)
library(readxl)
library(haven)
```

Instalación manual desde RStudio: Tools > Install Packages, o desde la pestaña Packages mediante Install. Esta vía es útil para principiantes, pero conviene aprender también la instalación por código porque deja explícito qué necesita el proyecto.

Error frecuente · Escribir `library(tidyverse)` cuando el paquete todavía no está instalado produce un error. Instalar y cargar son acciones distintas.

H. Comentarios: explicar el porqué del código

Todo lo que aparece después del símbolo # en una línea se interpreta como comentario y no se ejecuta. Los comentarios no deben repetir obviedades; deben explicar la finalidad, decisiones o etapas del análisis.

```
# Cargar ventas porque será la base principal del análisis
ventas <- read_csv("datos/ventas.csv")

# Excluir registros con fecha ausente antes de construir indicadores diarios
ventas_limpias <- ventas |> filter(!is.na(fecha))
```

Buena práctica · Comenta bloques de trabajo: 01 configuración, 02 importación, 03 auditoría, 04 limpieza, 05 análisis, 06 exportación. Esto permite entender el script varios meses después.

I. Ayuda y documentación dentro de R

```
?mean
help(mean)
example(mean)
args(mean)
```

La documentación de R incluye descripción, argumentos, valor devuelto y ejemplos. Aprender a leer Help reduce la dependencia de memorizar comandos.

Truco: cuando conozcas una función pero no recuerdes sus argumentos, revisa `?funcion`. Consultar la documentación es parte normal del trabajo profesional.

SESIÓN 1

Pensar en R

Objetos, lógica de programación, símbolos, funciones, tipos de datos, vectores, NA y data frames

Idea central · Antes de modelar, debes comprender cómo R representa la información y cómo documentar cada paso en un script reproducible.

1. La lógica básica de programación

Programar no significa necesariamente construir software complejo. En análisis de datos significa traducir una tarea a una secuencia clara de instrucciones. Una forma básica de visualizarlo es: entrada → transformación → salida.

Etapas	Pregunta	Ejemplo en análisis de datos
Entrada	¿Qué información recibe el proceso?	Una base CSV de empleo.
Transformación	¿Qué debo hacer con ella?	Filtrar, corregir, resumir, construir variables.
Salida	¿Qué deseo obtener?	Tabla, gráfico, base limpia o modelo.

Regla mental · Lee el código de izquierda a derecha y de arriba hacia abajo. Pregunta siempre: ¿qué objeto entra?, ¿qué función se aplica?, ¿qué objeto o resultado sale?

2. ¿Qué es un objeto en R?

Definición · Un objeto es una estructura que R mantiene en memoria. Normalmente lo referenciamos mediante un nombre para poder recuperarlo, transformarlo o utilizarlo después.

```
edad <- 35
region <- "Lima"
formal <- TRUE
```

El operador <- se lee como “asignar a”. Por ejemplo, edad <- 35 significa: guarda el valor 35 en un objeto llamado edad. El objeto aparecerá en Environment mientras exista en la sesión.

Objeto didáctico	Ejemplo	Clase / idea	Para qué sirve
Número	edad <- 35	numeric	Mediciones y operaciones aritméticas.
Texto	region <- "Lima"	character	Nombres, etiquetas, categorías en texto.
Condición lógica	formal <- TRUE	logical	Representar verdadero/falso y construir filtros.

Objeto didáctico	Ejemplo	Clase / idea	Para qué sirve
Vector	<code>ingreso <- c(1200, 1800, 2200)</code>	vector	Reunir varios valores del mismo tipo.
Tabla	<code>empleo <- data.frame(...)</code>	data.frame	Organizar observaciones en filas y variables en columnas.
Gráfico	<code>g <- ggplot(...) + geom_point()</code>	ggplot	Guardar una visualización para imprimirla o modificarla.
Modelo	<code>m <- lm(y ~ x, data = datos)</code>	lm	Guardar un modelo estimado y todos sus componentes.

Idea clave · En R casi todo puede tratarse como un objeto. Esto explica por qué puedes guardar una tabla, un gráfico o un modelo con un nombre y después pedirle a R que lo resuma, modifique o exporte.

```
ls()      # lista objetos creados en el entorno actual
class(edad) # clase del objeto
typeof(edad) # tipo interno de almacenamiento
```

3. Nombres de objetos: reglas y recomendaciones

Mejor	Evitar	Razón
<code>ingreso_mensual</code>	Ingreso Mensual	Los espacios obligan a usar comillas especiales y complican el código.
<code>fecha_venta</code>	<code>fv1</code>	Los nombres descriptivos facilitan leer el script.
<code>ventas_2026</code>	<code>2026_ventas</code>	Un nombre de objeto no debe comenzar con un número.
<code>precio_netto</code>	<code>precio.netto</code>	Ambos funcionan, pero conviene elegir una convención y mantenerla.

Recomendación · Usa snake_case: palabras en minúsculas separadas por guion bajo. Ejemplo: `ingreso_mensual`, `gasto_pc_alimentos`, `tasa_informalidad`.

4. Símbolos y operadores que debes reconocer

Símbolo	Lectura / función	Ejemplo
<code><-</code>	asignación	<code>x <- 10</code>

Símbolo	Lectura / función	Ejemplo
+ - * /	operaciones aritméticas	ingreso * 12
^	potencia	experiencia^2
==	igual a (comparación)	region == "Lima"
!=	diferente de	sexo != "Mujer"
> < >= <=	comparaciones numéricas	edad >= 18
&	Y lógico	edad >= 18 & edad <= 65
	O lógico	region == "Lima" region == "Callao"
!	negación	!is.na(ingreso)
\$	acceder a columna por nombre	datos\$ingreso
[]	indexación / selección	x[1:3]
[[]]	extraer un componente	datos[["ingreso"]]
:	crear secuencia	1:10
~	relación en una fórmula	ingreso ~ educacion
>	pipe nativo	datos > filter(...)
#	comentario	# limpiar duplicados

Error frecuente · No confundas = con ==. En una comparación se utiliza ==. Escribir region = "Lima" dentro de una condición no significa "preguntar si es Lima".

5. Operaciones y funciones

Una operación combina valores mediante operadores. Una función encapsula un procedimiento y se invoca escribiendo su nombre seguido de paréntesis. Los valores que recibe se llaman argumentos.

```
5 + 2
10 - 3
6 * 12
100 / 4
2^3

sqrt(81)
log(100)
round(3.14159, digits = 2)
```

Cómo leer una función · `round(3.14159, digits = 2)` se lee: aplica round al valor 3.14159 y utiliza 2 dígitos decimales. Los argumentos pueden ir por posición o por nombre.

6. Tipos de datos básicos

El tipo de dato determina qué operaciones tienen sentido. No puedes sumar directamente "Lima" + 2, y una fecha almacenada como texto no se comporta igual que una fecha reconocida por R.

Tipo	Ejemplo	Qué representa
numeric / double	1250.50	Números reales; es el tipo numérico habitual.
integer	25L	Enteros almacenados explícitamente.
character	"Lima"	Texto. Siempre entre comillas.
logical	TRUE / FALSE	Condiciones lógicas.
factor	<code>factor(c("Bajo", "Alto"))</code>	Categorías con niveles; muy útil en modelos.
Date	<code>as.Date("2026-08-14")</code>	Fechas con comportamiento cronológico.

```
edad <- 35
region <- "Lima"
formal <- TRUE
```

```
class(edad)
class(region)
class(formal)
```

Regla · Las comillas indican texto. TRUE y FALSE no llevan comillas cuando deseas valores lógicos.

7. Vectores: una columna de valores

Definición · Un vector es una estructura unidimensional que reúne varios elementos. En un vector atómico, los elementos comparten un tipo básico.

```
ingreso <- c(1250, 1800, 2300, 950, 3200, 2700, 1600, 4100, NA)
edad <- c(22, 28, 35, 41, 30, 50, 26, 38, 31)
```

```
length(ingreso)
ingreso[1]
ingreso[2:5]
ingreso[c(1, 3, 8)]
```


Los corchetes seleccionan posiciones. También puedes colocar dentro una condición lógica; R conservará las posiciones donde la condición sea TRUE.

```
ingreso[ingreso > 2000]  
edad[ingreso > 2000]
```

Consejo · Antes de usar un vector en una función, revisa `length()`, `class()` y, si hay dudas, `head()` o `print()`.

8. Condiciones lógicas: la base de los filtros

```
ingreso > 2000  
edad >= 18 & edad <= 65  
region == "Lima"  
formal == TRUE
```

Una comparación devuelve TRUE o FALSE para cada elemento. Esa salida lógica se utiliza después para seleccionar observaciones o crear nuevas variables.

Idea clave · `filter()` de dplyr y muchas operaciones de selección dependen de condiciones lógicas. Por eso comprender TRUE/FALSE es más importante que memorizar filtros.

9. NA: dato faltante

Definición · NA representa un valor no disponible. No significa cero, ni cadena vacía, ni “no aplica” automáticamente. Su interpretación depende de cómo se construyó la base.

```
is.na(ingreso)  
sum(is.na(ingreso))  
  
mean(ingreso)  
mean(ingreso, na.rm = TRUE)
```

Error frecuente · No busques faltantes con `ingreso == NA`. Las comparaciones con NA no se resuelven como TRUE/FALSE de forma ordinaria. Utiliza `is.na()`.

Nota metodológica · `na.rm = TRUE` excluye los NA del cálculo; no “soluciona” el dato faltante. La decisión de eliminar, imputar o recuperar información exige una justificación analítica.

10. Data frame: la estructura central del análisis

Definición · Un data frame es una tabla rectangular. En análisis aplicado, normalmente cada fila representa una observación y cada columna una variable. Las columnas pueden ser de tipos diferentes.

```
trabajadores <- data.frame(
  id = 1:8,
  sexo = c("Mujer", "Hombre", "Hombre", "Mujer", "Mujer", "Hombre", "Mujer", "Hombre"),
  edad = c(22, 28, 35, 41, 30, 50, 26, 38),
  educacion = c(12, 16, 14, 11, 18, 12, 16, 14),
  ingreso = c(1250, 1800, 2300, 950, 3200, 2700, 1600, 4100)
)

head(trabajadores)
dim(trabajadores)
names(trabajadores)
str(trabajadores)
summary(trabajadores)
```

Función	Pregunta que responde
head()	¿Cómo se ven las primeras observaciones?
dim()	¿Cuántas filas y columnas hay?
names()	¿Cómo se llaman las variables?
str() / glimpse()	¿Qué tipo y estructura tiene cada columna?
summary()	¿Qué resumen general presenta cada variable?

11. Selección de filas y columnas

```
trabajadores$ingreso
trabajadores[["ingreso"]]
trabajadores[1, ]
trabajadores[1:3, ]
trabajadores[trabajadores$ingreso > 2000, ]
```

En los corchetes de un data frame, la forma general es [filas, columnas]. Dejar un lado vacío significa “todas”. Por ejemplo `trabajadores[1, ]` devuelve la primera fila y todas las columnas.

12. Matrices: un puente conceptual hacia econometría

No necesitas dominar álgebra matricial en este curso. Sin embargo, conviene reconocer que muchos procedimientos econométricos se expresan en forma matricial. Una matriz contiene elementos de un mismo tipo y tiene filas y columnas.

```
X <- matrix(c(1,1,1,2,4,6), nrow = 3, ncol = 2)
y <- matrix(c(10,20,30), ncol = 1)
```

```
X
t(X)
t(X) %*% X
```

Puente econométrico · Más adelante verás expresiones como $y = X\beta + u$. Reconocer una matriz y una multiplicación matricial evita que esa notación resulte completamente nueva.

Ejercicios de la sesión 1

EJERCICIO 1.1 · Ingresos y filtros · Crea el vector ingreso mostrado en la sección 7. Calcula número de observaciones, cantidad de NA, media, mediana, máximo; obtén ingresos mayores a 2 000 y cuenta cuántos son. Explica por qué `mean(ingreso)` devuelve NA sin `na.rm = TRUE`.

EJERCICIO 1.2 · Primera base laboral · Construye trabajadores. Identifica filas y columnas; extrae ingreso de tres formas; filtra ingresos mayores a 2 000; calcula ingreso promedio de mujeres; identifica a la persona con mayor ingreso y crea `ingreso_anual = ingreso * 12`.

RETO · Escribe comentarios que separen tu script en CONFIGURACIÓN, OBJETOS, VECTORES, BASE y RESULTADOS. El objetivo es que otra persona pueda leer tu lógica sin preguntarte qué hiciste.

SESIÓN 2

De archivo bruto a base analítica

Importación, inspección, calidad de datos, dplyr, pipes y transformación reproducible

Idea central · Un buen análisis empieza por una base bien importada, auditada y transformada. El modelo no corrige una base mal construida.

1. Importar no es “abrir”: es crear un objeto

Cuando importas un archivo, R lee su contenido y crea un objeto en memoria. El archivo original permanece en disco. Esta diferencia permite conservar la fuente y trabajar sobre copias lógicas dentro del análisis.

Regla · Nombra el objeto según su rol: empleo, ventas, panel, contexto. Evita nombres como base1, base2, final_final2.

2. Rutas relativas dentro del Project

```
getwd()
list.files()
list.files("datos")
```

Si `getwd()` apunta a la raíz del Project, la ruta "datos/empleo_peru_bruto.csv" significa: entra a la carpeta datos que está dentro del proyecto y busca ese archivo.

Error frecuente · Si aparece “cannot open file” o “file does not exist”, revisa `getwd()`, `list.files()` y el nombre exacto del archivo antes de cambiar código al azar.

3. Importar CSV, Excel y Stata

```
library(tidyverse)
library(readxl)
library(haven)

empleo <- read_csv("datos/empleo_peru_bruto.csv")
empleo_xlsx <- read_excel("datos/empleo_peru_bruto.xlsx")
empleo_dta <- read_dta("datos/empleo_peru_bruto.dta")
```

Formato	Función	Observación
CSV	<code>readr::read_csv()</code>	Formato plano, ligero y ampliamente interoperable.
Excel	<code>readxl::read_excel()</code>	Permite elegir hoja y rango; revisa nombres y tipos importados.

Formato	Función	Observación
Stata	haven::read_dta()	Conserva información útil de formatos Stata y puede traer etiquetas.

Buena práctica · Usa el prefijo paquete::funcion cuando quieras dejar explícito de dónde viene una función, por ejemplo readr::read_csv(). No es obligatorio cuando el paquete ya está cargado, pero ayuda a documentar.

4. Auditoría inicial: nunca modelos inmediatamente después de importar

```
head(empleo)
tail(empleo)
dim(empleo)
names(empleo)
glimpse(empleo)
summary(empleo)
colSums(is.na(empleo))
```

Pregunta de control	Herramienta
¿Cuántas observaciones y variables hay?	dim(), nrow(), ncol()
¿Cómo se llaman las variables?	names()
¿Qué tipo tiene cada columna?	glimpse(), str()
¿Qué rangos aparecen?	summary(), min(), max(), quantile()
¿Cuántos faltantes hay?	colSums(is.na())
¿Qué categorías existen?	unique(), sort(unique())

Regla · La auditoría es descriptiva y diagnóstica. Primero identifica problemas; después decide cómo corregirlos. No limpies automáticamente sin entender qué representa la variable.

5. Identificadores y duplicados

Un identificador permite distinguir unidades de análisis. Que un ID aparezca dos veces no siempre es un error: podría indicar mediciones repetidas, transacciones múltiples o panel. Debes conocer la unidad de análisis antes de eliminar.

```
empleo |>
  count(id_persona) |>
  filter(n > 1)
```

Pregunta obligatoria · ¿Cada fila debe representar una persona única, una persona-año, una venta, un producto-tienda-día u otra unidad? La respuesta define qué significa “duplicado”.

6. Texto inconsistente y estandarización

```
sort(unique(empleo$region))

empleo <- empleo |>
mutate(region = str_to_title(str_trim(region)))
```

"Lima", "lima" y "Lima " pueden referirse al mismo lugar, pero son cadenas diferentes. Esto afecta agrupaciones y joins. `str_trim()` quita espacios extremos y `str_to_title()` estandariza capitalización.

7. El pipe |> : leer una secuencia de transformaciones

Definición · El pipe nativo `|>` toma el resultado de la expresión de la izquierda y lo pasa a la función de la derecha. Permite leer una secuencia como una cadena de pasos.

```
empleo |>
filter(region == "Lima") |>
select(id_persona, sexo, ingreso_mensual) |>
arrange(desc(ingreso_mensual))
```

Léelo como una frase: toma empleo, filtra Lima, selecciona tres variables y ordena de mayor a menor ingreso.

Consejo · Evita pipes excesivamente largos. Si una transformación produce un objeto importante, asígnalo a un nombre y continúa desde allí.

8. Verbos fundamentales de dplyr

Verbo	Pregunta mental	Ejemplo
<code>select()</code>	¿Qué columnas necesito?	<code>select(region, sexo, ingreso_mensual)</code>
<code>filter()</code>	¿Qué filas cumplen una condición?	<code>filter(edad >= 18)</code>
<code>arrange()</code>	¿Cómo quiero ordenar?	<code>arrange(desc(ingreso_mensual))</code>
<code>rename()</code>	¿Qué nombre debe cambiar?	<code>rename(ingreso = ingreso_mensual)</code>
<code>mutate()</code>	¿Qué variable nueva o transformada necesito?	<code>mutate(ln_ingreso = log(ingreso_mensual))</code>
<code>distinct()</code>	¿Qué filas duplicadas deseo identificar/conservar?	<code>distinct(id_persona, .keep_all = TRUE)</code>

9. Limpieza reproducible

```
empleo_limpio <- empleo |>
  distinct(id_persona, .keep_all = TRUE) |>
  mutate(
    region = str_to_title(str_trim(region)),
    edad = if_else(edad >= 18 & edad <= 80, edad, NA_real_),
    ingreso_mensual = if_else(ingreso_mensual > 0, ingreso_mensual, NA_real_)
  )
```

El objetivo no es “hacer desaparecer errores” sino transformar reglas explícitas en código. Si se considera imposible una edad fuera de 18–80 para la muestra analítica, esa decisión debe quedar escrita y ser revisable.

Nota metodológica · Convertir un valor imposible a NA es una decisión de depuración, no una verdad universal. El rango válido depende del diseño de la investigación o de la lógica del negocio.

10. Crear variables para el análisis

```
empleo_limpio <- empleo_limpio |>
  mutate(
    ingreso_anual = ingreso_mensual * 12,
    ln_ingreso = log(ingreso_mensual),
    mujer = if_else(sexo == "Mujer", 1, 0),
    formal_dummy = if_else(formal == "Si", 1, 0)
  )
```

Regla · Crea variables mediante código y conserva el nombre de la fuente original. Así puedes reconstruir la base desde cero cuando recibas una nueva versión de los datos.

Ejercicios de la sesión 2

EJERCICIO 2.1 · Auditoría de la base · Importa datos/empleo_peru_bruto.csv. Registra dimensiones, NA por columna, identificadores duplicados, categorías de región, edades fuera de 18–80 e ingresos ≤ 0 . Documenta los hallazgos con comentarios.

EJERCICIO 2.2 · Base para un modelo de ingreso · Crea empleo_limpio. Conserva variables relevantes, elimina del análisis filas sin ingreso/edad/educación válidos, crea ln_ingreso, experiencia2, mujer y formal_dummy, ordena por ingreso y muestra los cinco mayores.

RETO · Escribe el proceso de limpieza como un pipeline legible. Luego separa el pipeline en dos objetos si consideras que mejora la comprensión. Explica tu decisión.

SESIÓN 3

Convertir datos en información

Estadística descriptiva, agrupación, joins, visualización y correlación

Idea central · Describir y visualizar precede a modelar: primero entiende la distribución y las relaciones; después formula preguntas econométricas.

1. ¿Por qué describir antes de modelar?

Un modelo puede producir coeficientes incluso cuando la base contiene problemas o relaciones atípicas. La exploración descriptiva permite conocer magnitudes, dispersión, categorías, valores extremos y patrones que deben entenderse antes de cualquier inferencia.

Regla · No interpretes un coeficiente de regresión sin haber visto antes cómo se distribuyen las variables principales.

2. Estadísticos descriptivos esenciales

```
mean(empleo_limpio$ingreso_mensual, na.rm = TRUE)
median(empleo_limpio$ingreso_mensual, na.rm = TRUE)
sd(empleo_limpio$ingreso_mensual, na.rm = TRUE)
quantile(empleo_limpio$ingreso_mensual, c(.25, .5, .75), na.rm = TRUE)
```

Estadístico	Interpretación conceptual
Media	Promedio aritmético; sensible a valores extremos.
Mediana	Valor central al ordenar datos; resistente a extremos comparada con la media.
Desviación estándar	Magnitud típica de dispersión respecto a la media.
Cuartiles	Puntos que dividen los datos ordenados en cuatro partes.
Mínimo / máximo	Extremos observados; útiles para detectar rangos inesperados.

3. Frecuencias y proporciones

```
table(empleo_limpio$sexo)
prop.table(table(empleo_limpio$sexo))
```

Las frecuencias son esenciales para variables categóricas. Una proporción permite comparar composición aun cuando cambie el tamaño total de la muestra.

4. group_by() + summarise(): de microdatos a indicadores

```
resumen_region <- empleo_limpio |>
  group_by(region) |>
  summarise(
    n = n(),
    ingreso_promedio = mean(ingreso_mensual, na.rm = TRUE),
    ingreso_mediano = median(ingreso_mensual, na.rm = TRUE),
    educacion_promedio = mean(educacion_anios, na.rm = TRUE),
    formalidad_pct = mean(formal == "Si", na.rm = TRUE) * 100,
    .groups = "drop"
  ) |>
  arrange(desc(ingreso_promedio))
```

Cómo leerlo · group_by(region) define el nivel de agrupación. summarise() reduce cada grupo a una fila con indicadores. Este patrón será muy útil para construir paneles regionales, departamentales o empresariales.

5. Llaves y joins: unir información sin perder control

Definición · Una llave es una variable o combinación de variables que permite identificar cómo se corresponden filas entre dos bases.

```
contexto <- read_csv("datos/contexto_regional_2025.csv")
```

```
base_regional <- resumen_region |>
  left_join(contexto, by = "region")
```

Join	Qué conserva	Uso típico
left_join(x, y)	Todas las filas de x y coincidencias de y.	Agregar variables a una base principal sin perder sus filas.
inner_join(x, y)	Solo coincidencias en ambas bases.	Trabajar exclusivamente con casos comunes.
full_join(x, y)	Todas las filas de x e y.	Auditar cobertura global de dos fuentes.
anti_join(x, y)	Filas de x sin pareja en y.	Diagnosticar llaves que no encontraron coincidencia.

```
# Diagnóstico de regiones sin pareja
resumen_region |>
  anti_join(contexto, by = "region")
```

Error frecuente · Un join puede multiplicar filas si la llave no es única donde esperabas una relación 1:1. Antes de unir, cuenta duplicados de la llave en ambas bases.

6. Visualización con ggplot2: datos + estética + geometría

ggplot2 construye gráficos por capas. El objeto inicial define datos y estética; las geometrías indican cómo representar visualmente esas variables. Puedes guardar el gráfico en un objeto y modificarlo después.

```
g_ingreso <- ggplot(empleo_limpio, aes(x = ingreso_mensual)) +
  geom_histogram(bins = 15) +
  labs(title = "Distribución del ingreso mensual",
        x = "Ingreso mensual", y = "Frecuencia")

g_ingreso
```

```
ggplot(empleo_limpio, aes(x = formal, y = ingreso_mensual)) +
  geom_boxplot() +
  labs(title = "Ingreso según formalidad", x = "Formalidad", y = "Ingreso mensual")
```

```
ggplot(empleo_limpio, aes(x = educacion_anios, y = ingreso_mensual)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Educación e ingreso", x = "Años de educación", y = "Ingreso mensual")
```

Gráfico	Pregunta principal
Histograma	¿Cómo se distribuye una variable numérica?
Boxplot	¿Cómo cambia la distribución entre grupos?
Dispersión	¿Cómo se relacionan visualmente dos variables numéricas?
Barras	¿Cómo comparar magnitudes por categorías?

Interpretación responsable · Un gráfico muestra patrones de la muestra; no prueba causalidad. Una pendiente visual positiva no significa que aumentar X cause automáticamente un aumento en Y.

7. Correlación: dirección e intensidad lineal

```
cor(empleo_limpio$educacion_anios,
    empleo_limpio$ingreso_mensual,
    use = "complete.obs")
```

La correlación resume asociación lineal entre dos variables numéricas. Se encuentra entre -1 y 1; su magnitud no reemplaza el análisis gráfico y no controla otras variables.

Regla · Correlación no implica causalidad. Esta frase no es un formalismo: es una advertencia sobre identificación.

Ejercicios de la sesión 3

EJERCICIO 3.1 · Perfil laboral por grupos · Agrupa empleo_limpio por sexo y formalidad. Calcula n, ingreso promedio y educación promedio. Ordena de mayor a menor ingreso y escribe dos conclusiones descriptivas sin lenguaje causal.

EJERCICIO 3.2 · Contexto regional · Importa contexto_regional_2025.csv, verifica que region sea única, une con resumen_region, usa anti_join() como diagnóstico y grafica pobreza_pct frente a ingreso_promedio.

RETO · Antes del left_join(), crea una comprobación que detecte si contexto tiene más de una fila por región. Explica por qué esa prueba protege el análisis.

SESIÓN 4**Preparar variables para econometría**

Logaritmos, dummies, términos cuadráticos, factores, interacciones, panel, rezagos y fórmulas

Idea central · Transformar variables tiene una razón económica o estadística. No se crean variables solo para buscar significancia.

1. Transformar no es maquillar: cada variable necesita una razón

Una transformación cambia la escala o representación de una variable. Puede responder a teoría económica, interpretación, distribución o estructura temporal. No debe usarse solo para “hacer significativo” un coeficiente.

Regla · Primero formula por qué la transformación tiene sentido. Después escribe el código.

2. Logaritmos

El logaritmo natural se utiliza con frecuencia en variables positivas y de gran dispersión. También facilita interpretar ciertas relaciones en términos proporcionales o porcentuales cuando el modelo está correctamente especificado.

```
base <- empleo_limpio |>
  filter(ingreso_mensual > 0) |>
  mutate(ln_ingreso = log(ingreso_mensual))
```

Error frecuente · $\log(0)$ produce $-\text{Inf}$ y $\log()$ de un valor negativo no genera un dato real utilizable en este contexto. Revisa el dominio antes de transformar.

3. Variables dummy

Definición · Una dummy codifica una condición mediante 1 y 0. Su interpretación depende de qué categoría fue definida como 1 y cuál actúa como referencia.

```
base <- base |>
  mutate(
    mujer = if_else(sexo == "Mujer", 1, 0),
    formal_dummy = if_else(formal == "Si", 1, 0)
  )
```

Regla · El nombre debe revelar qué significa 1. mujer es más informativo que sexo_dummy si 1 corresponde a Mujer.

4. Términos cuadráticos

Un término cuadrático permite que la relación entre una variable y el resultado no sea estrictamente lineal. En salarios, por ejemplo, se utiliza con frecuencia `experiencia` y `experiencia2` para representar rendimientos que cambian con los años acumulados.

```
base <- base | >
mutate(experiencia2 = experiencia_anios^2)
```

Nota · No interpretes `experiencia2` aisladamente cuando `experiencia` y `experiencia2` están en el mismo modelo. El efecto marginal depende de ambos coeficientes.

5. Factores y categoría de referencia

Un factor representa categorías y niveles. En modelos, R utiliza contrastes para comparar niveles respecto a una categoría de referencia.

```
base <- base | >
mutate(
  sexo = factor(sexo),
  sector = factor(sector)
)

levels(base$sexo)
base$sexo <- relevel(base$sexo, ref = "Hombre")
```

Buena práctica · Define conscientemente la categoría de referencia cuando la interpretación económica depende de ella. No dejes esta decisión completamente a un orden alfabético accidental.

6. Interacciones: cuando una relación depende de otra variable

Una interacción representa la posibilidad de que la asociación entre X e Y cambie según el nivel o categoría de Z. En fórmulas de R, `x*z` incluye x, z y su interacción; `x:z` incluye solo el término de interacción.

```
# Ejemplo de sintaxis; la interpretación formal se verá en econometría
lm(ln_ingreso ~ educacion_anios * sexo, data = base)
```

Nota · Una interacción no debe agregarse por rutina. Se justifica cuando existe una hipótesis sobre heterogeneidad del efecto o asociación.

7. ¿Qué es una base de panel?

Definición · Un panel combina una dimensión transversal (unidad) y una dimensión temporal. Ejemplo: región-año, empresa-trimestre o persona-mes.

```
panel <- read_csv("datos/panel_regional_2022_2025.csv")

panel |> count(region, anio) |> filter(n > 1)
panel |> count(region) |> arrange(n)
```

La combinación de identificadores debe corresponder a la unidad de observación. En un panel región-año, region + anio debería identificar cada fila si el panel está construido a ese nivel.

Regla · Antes de crear rezagos, ordena correctamente por unidad y tiempo. Un lag sin agrupación puede tomar el valor de otra región.

8. Rezagos y diferencias

```
panel <- panel |>
  arrange(region, anio) |>
  group_by(region) |>
  mutate(
    inversion_lag = lag(inversion_publica_pc),
    delta_pobreza = pobreza_pct - lag(pobreza_pct)
  ) |>
  ungroup()
```

lag() desplaza una variable dentro del orden definido. La primera observación de cada grupo no tiene valor previo, por lo que el rezago será NA. Una diferencia mide cambio entre periodos consecutivos según la forma definida.

9. Fórmulas de modelo en R

Definición · La sintaxis $y \sim x_1 + x_2$ describe una relación de modelamiento: la variable a la izquierda es la respuesta y las de la derecha son predictores. El símbolo $\sim$ no significa igualdad matemática.

```
ingreso_mensual ~ educacion_anios
ln_ingreso ~ educacion_anios + experiencia_anios + experiencia2 + sexo
```

Sintaxis	Lectura
$y \sim x$	Y modelado con X.
$y \sim x_1 + x_2$	Y con dos predictores.
$y \sim x * z$	Incluye x, z y x:z.
$y \sim x + I(x^2)$	Incluye x y su cuadrado escrito dentro de la fórmula.
$y \sim .$	Y con todas las demás variables disponibles en data, con cautela.

Ejercicios de la sesión 4

EJERCICIO 4.1 · Transformaciones · En `empleo_limpio` crea `ln_ingreso`, `experiencia2`, `mujer` y `formal_dummy`. Convierte `sector` a factor y revisa sus niveles. Explica en una frase el sentido de cada transformación.

EJERCICIO 4.2 · Estructura de panel · Importa `panel_regional_2022_2025.csv`. Verifica duplicados `región-año`, ordena por `región` y `año`, crea un rezago de inversión y una diferencia de pobreza. Revisa los NA generados por el rezago.

RETO · Explica por qué sería incorrecto calcular `lag(inversion_publica_pc)` sin `group_by(region)` si varias regiones están apiladas en la misma tabla.

SESIÓN 5**Del dato bruto a una primera regresión**

Pipeline reproducible, `lm()`, objetos de modelo, interpretación básica, diagnóstico visual y exportación

Idea central · El resultado final de un análisis reproducible es un flujo completo: importar, limpiar, explorar, transformar, modelar, interpretar y exportar.

1. El pipeline completo

Un análisis profesional no empieza en `lm()`. La regresión es una etapa dentro de una cadena. Si las etapas anteriores no son reproducibles, tampoco lo será el resultado final.

```
# 01. Configuración
library(tidyverse)

# 02. Importación
empleo <- read_csv("datos/empleo_peru_bruto.csv")

# 03. Auditoría y limpieza
# ...

# 04. Transformaciones
# ...

# 05. Descriptivos y gráficos
# ...

# 06. Modelo
# ...

# 07. Exportación
# ...
```

Regla · El script debe poder ejecutarse desde una sesión limpia y reconstruir los resultados sin depender de objetos creados manualmente antes.

2. ¿Qué hace `lm()`?

`lm()` ajusta modelos lineales. En este curso se utiliza como puente hacia econometría: el objetivo es reconocer la sintaxis, guardar el modelo como objeto y aprender a inspeccionar sus componentes. La inferencia formal y los supuestos pertenecen al siguiente nivel.

```
modelo1 <- lm(ingreso_mensual ~ educacion_anios, data = empleo_limpio)
summary(modelo1)
```

Cómo leer el código · `modelo1` es el nombre del objeto. `lm()` es la función. `ingreso_mensual ~ educacion_anios` es la fórmula. `data = empleo_limpio` indica en qué tabla buscar las variables.

3. Regresión múltiple: la fórmula crece, la lógica se mantiene

```
modelo2 <- lm(
  ln_ingreso ~ educacion_anios + experiencia_anios + experiencia2 + sexo,
  data = base
)

summary(modelo2)
```

Agregar variables no convierte automáticamente el modelo en causal. Una regresión múltiple permite condicionar por predictores incluidos, pero la interpretación depende del diseño, supuestos y problema de investigación.

Advertencia econométrica · No uses “significativo” como sinónimo de “importante” ni “causal”. En el curso posterior se estudiará inferencia, robustez, diagnóstico y estrategias de identificación.

4. El modelo también es un objeto

```
class(modelo2)
coef(modelo2)
fitted(modelo2)
residuals(modelo2)
summary(modelo2)
```

Función	Qué recupera
coef()	Coeficientes estimados.
fitted()	Valores ajustados por el modelo.
residuals()	Residuos observados.
summary()	Resumen con coeficientes y estadísticas del ajuste.
predict()	Predicciones para observaciones existentes o nuevas, según argumentos.

Idea clave · Guardar el modelo te permite no repetir la estimación cada vez que quieras extraer coeficientes, residuos, predicciones o gráficos de diagnóstico.

5. Predicción didáctica

```
nuevo <- data.frame(  
  educacion_anios = 16,  
  experiencia_anios = 8,  
  experiencia2 = 8^2,  
  sexo = factor("Hombre", levels = levels(base$sexo))  
)  
  
predict(modelo2, newdata = nuevo)
```

Nota · Una predicción requiere que newdata contenga variables compatibles con las que utilizó el modelo, incluidos niveles de factores.

6. Diagnóstico visual introductorio

```
par(mfrow = c(2, 2))  
plot(modelo2)  
par(mfrow = c(1, 1))
```

Los gráficos de diagnóstico permiten observar patrones en residuos, dispersión y observaciones influyentes. En este curso solo se introducen visualmente; su interpretación formal se estudiará en econometría.

7. Exportar resultados

```
write_csv(empleo_limpio, "resultados/empleo_limpio.csv")  
  
ggsave(  
  filename = "graficos/educacion_ingreso.png",  
  width = 8, height = 5, dpi = 300  
)
```

Buena práctica · No sobrescribas datos brutos con bases procesadas. Guarda los resultados en una carpeta separada y conserva el script que los generó.

Proyecto integrador final

Caso. Se desea preparar una base laboral para estudiar la relación entre ingreso, educación, experiencia, sexo y formalidad. El objetivo principal es demostrar que puedes construir todo el flujo sin depender de Excel para limpiar manualmente.

11. Abre el Project del curso y verifica getwd().
12. Carga los paquetes y confirma que la carpeta datos sea visible.
13. Importa empleo_peru_bruto.csv.
14. Audita dimensiones, nombres, tipos, NA, rangos, categorías y duplicados.
15. Construye empleo_limpio con reglas explícitas.
16. Crea ingreso_anual, ln_ingreso, experiencia2, mujer y formal_dummy.

17. Genera un resumen por región y un resumen por sexo/formalidad.
18. Construye al menos un histograma, un boxplot y una dispersión.
19. Importa contexto regional y realiza un `left_join()` controlado.
20. Estima un modelo lineal simple y uno múltiple como ejercicio introductorio.
21. Extrae coeficientes y residuos del modelo múltiple.
22. Exporta la base limpia y un gráfico.
23. Comprueba que todo pueda ejecutarse nuevamente desde una sesión limpia.

Criterio de logro · Otro estudiante debería poder abrir tu Project, ejecutar el script de arriba hacia abajo y obtener los mismos archivos de salida sin preguntarte qué hiciste manualmente.

Hoja rápida de comandos

Necesidad	Comando / patrón
Directorio actual	<code>getwd()</code>
Cambiar directorio (uso didáctico)	<code>setwd("D:/carpeta")</code>
Listar archivos	<code>list.files()</code>
Instalar paquete	<code>install.packages("paquete")</code>
Cargar paquete	<code>library(paquete)</code>
Ayuda	<code>?funcion</code>
Crear objeto	<code>x <- valor</code>
Crear vector	<code>c(...)</code>
Tipo / clase	<code>typeof(x), class(x)</code>
Faltantes	<code>is.na(x), sum(is.na(x))</code>
Inspeccionar tabla	<code>head(), dim(), names(), glimpse(), summary()</code>
Seleccionar columnas	<code>select()</code>
Filtrar filas	<code>filter()</code>
Ordenar	<code>arrange()</code>
Crear variable	<code>mutate()</code>
Agrupar	<code>group_by()</code>
Resumir	<code>summarise()</code>
Unir bases	<code>left_join()</code>
Gráfico	<code>ggplot() + geom_*()</code>
Correlación	<code>cor()</code>
Modelo lineal	<code>lm(y ~ x1 + x2, data = datos)</code>
Exportar CSV	<code>write_csv()</code>

Errores frecuentes y cómo diagnosticarlos

Mensaje / situación	Qué revisar primero
object not found	¿El objeto se creó? ¿Está bien escrito? ¿Mayúsculas/minúsculas? Revisa ls().
file does not exist / cannot open	Revisa getwd(), list.files(), carpeta y nombre exacto.
could not find function	¿Cargaste el paquete con library()? ¿La función pertenece a otro paquete?
there is no package called...	Instala el paquete antes de cargarlo.
NAs introduced by coercion	Se intentó convertir texto no numérico a número; revisa los valores.
replacement has ... rows	La longitud del objeto asignado no coincide con el número de filas.
join multiplicó observaciones	Revisa duplicados en las llaves de ambas bases.
mean() devuelve NA	Revisa si existen NA y decide si corresponde na.rm = TRUE.
log() genera -Inf / NaN	Revisa ceros y negativos antes de transformar.
modelo usa menos observaciones	lm() omite filas con faltantes en variables del modelo por defecto; verifica datos completos.

Glosario mínimo

Término	Definición breve
Argumento	Valor que se entrega a una función.
Asignación	Acción de guardar un valor u objeto bajo un nombre.
Clase	Información que guía cómo se comportan ciertas funciones con un objeto.
Data frame	Tabla rectangular con filas y columnas, cuyas columnas pueden tener tipos distintos.
Directorio de trabajo	Carpeta que R utiliza como referencia para rutas relativas.
Factor	Representación de una variable categórica mediante niveles.
Función	Procedimiento reutilizable que recibe argumentos y devuelve un resultado.
Llave	Variable o combinación de variables usada para emparejar registros entre bases.

Término	Definición breve
NA	Valor no disponible.
Objeto	Estructura almacenada y manipulada por R.
Pipe	Operador que encadena el resultado de un paso con el siguiente.
Project	Contexto de trabajo de RStudio asociado a una carpeta y archivo .Rproj.
Script	Archivo de texto con instrucciones de R.
Vector	Estructura unidimensional de elementos.

Referencias y recursos oficiales de consulta

Este manual se apoya en la documentación oficial del R Core Team y de Posit para los conceptos de lenguaje R, objetos, tipos, interfaz de RStudio, Projects, paquetes y R Markdown. Se recomienda consultar las versiones vigentes de los siguientes recursos:

- R Core Team. An Introduction to R.
- R Core Team. The R Language Definition.
- Posit. RStudio IDE User Guide.
- Posit. RStudio Projects.
- Posit. Pane Layout / RStudio interface documentation.
- Posit. R Markdown documentation.

Cierre · Aprender R no consiste en memorizar una lista de comandos. Consiste en comprender la lógica de objetos, funciones, datos y transformaciones hasta poder construir un flujo claro que otra persona pueda reproducir.